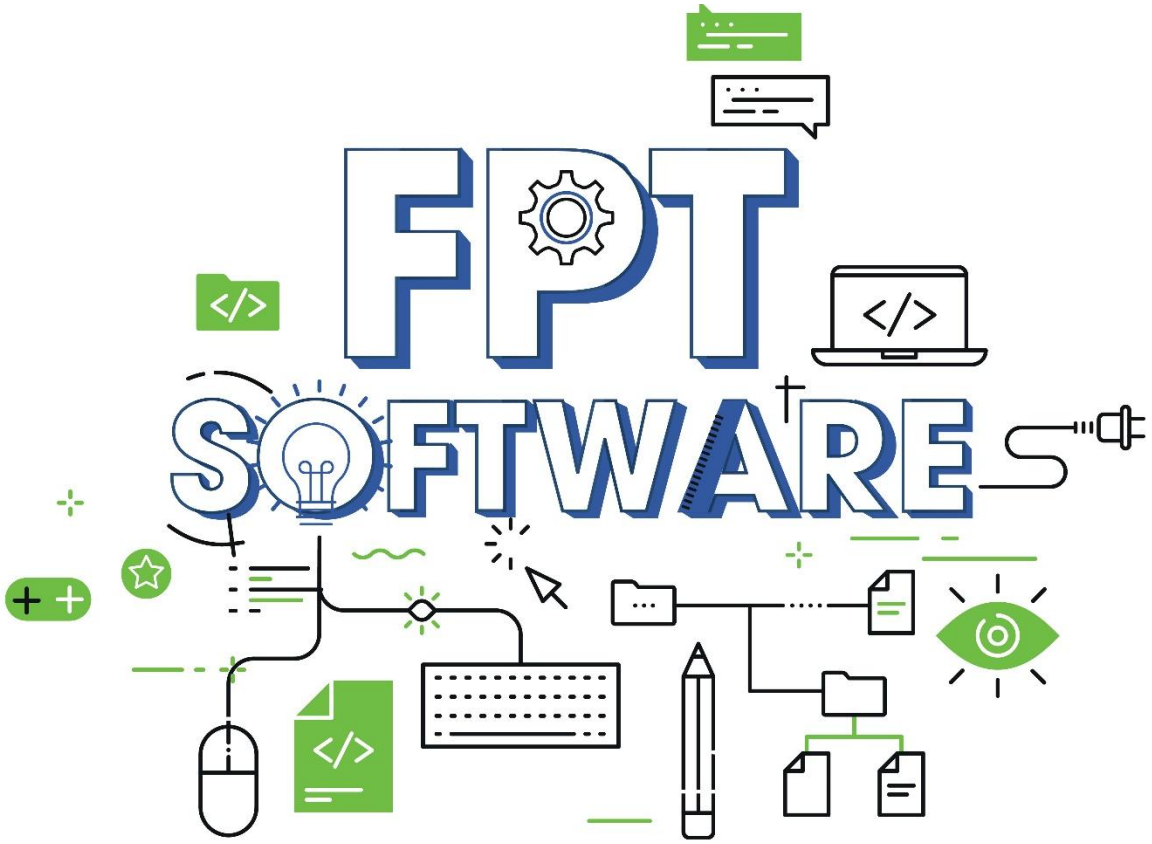
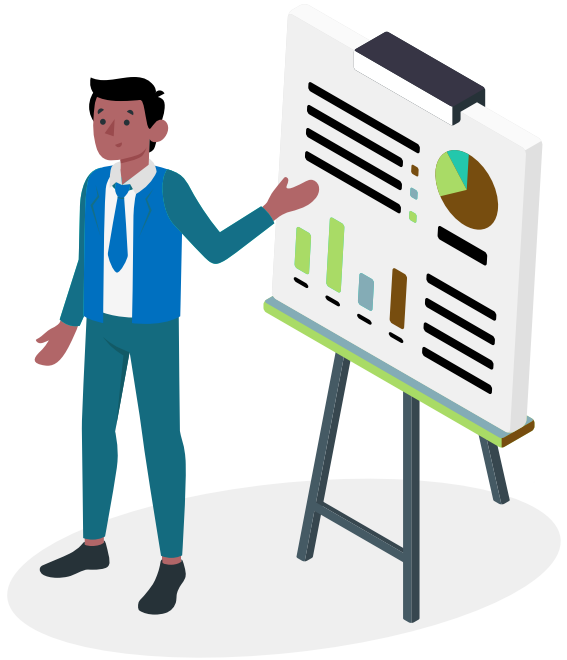
Basic Android

Android Notification



Agenda

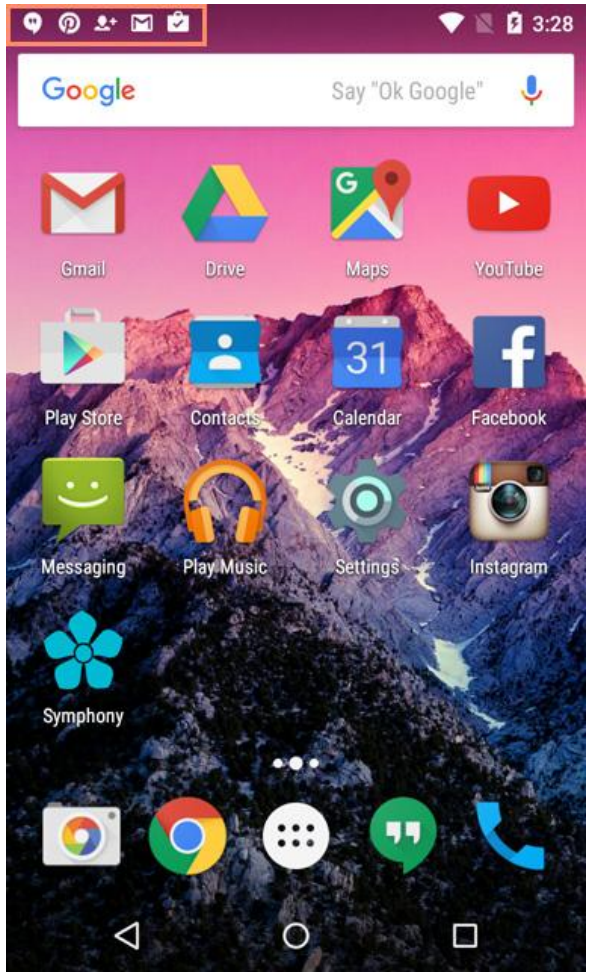
1. Introduction to Notification
2. Role of Notification
3. Types of Notification in Android
4. The structure of the notification
5. Creating, Managing a Notification

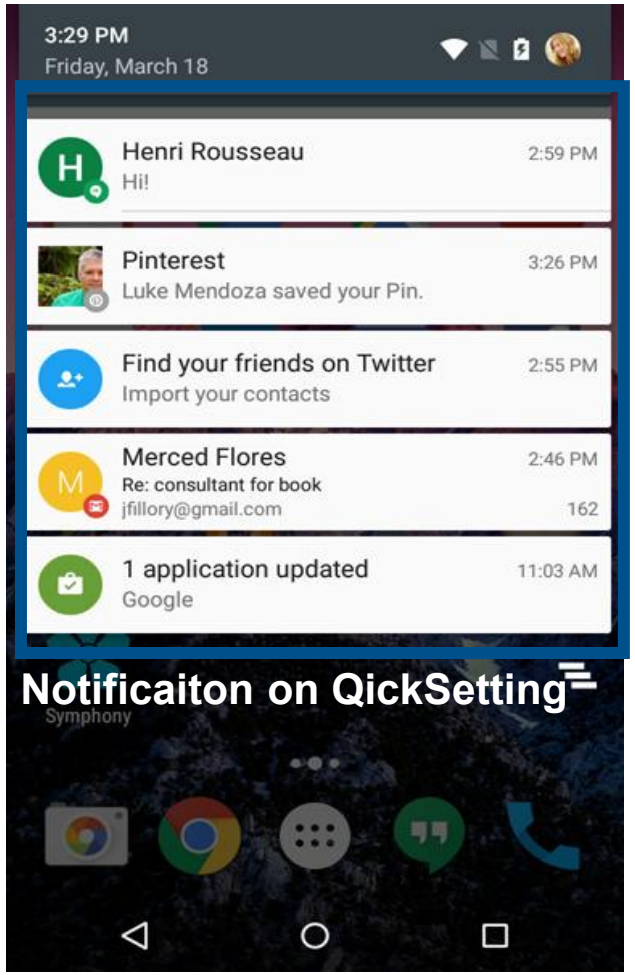


1. Introduction to notification

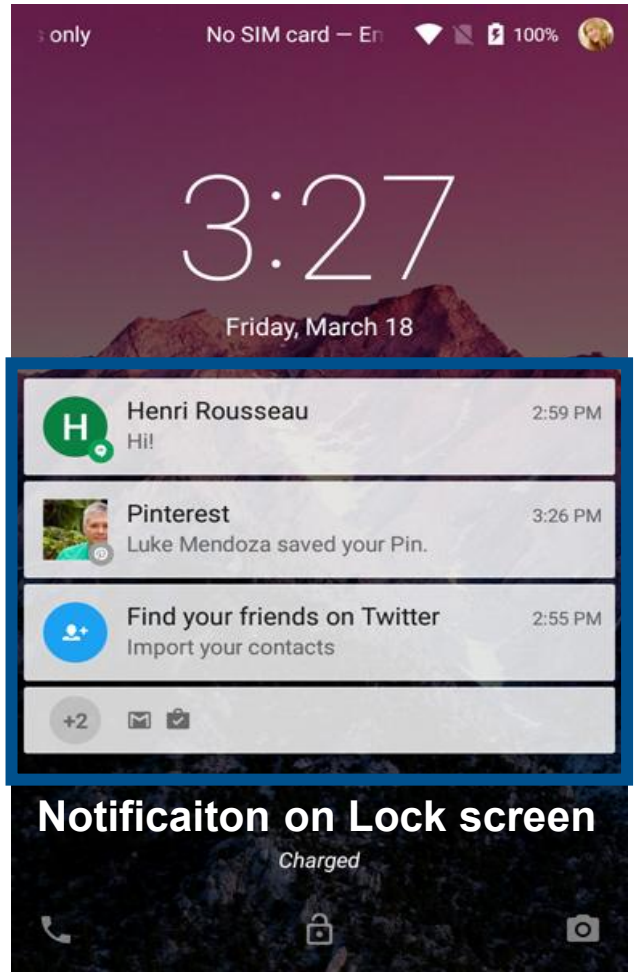
A notification is a message that Android displays outside your app's UI to provide the user with reminders, communication from other people, or other timely information from your app. Users can tap the notification to open your app or take an action directly from the notification.

1. Introduction to notification





Notificaiton on QickSetting



Notificaiton on Lock screen

2. Role of Notification

- Keep users engaged: Notifications help keep your app on users' radar and maintain their interest.
- Notify about events: Send notifications about important events or news to users as soon as they occur.
- Increase interaction: Allow users to take actions directly from the notification without opening the app.

3. Role of Notification

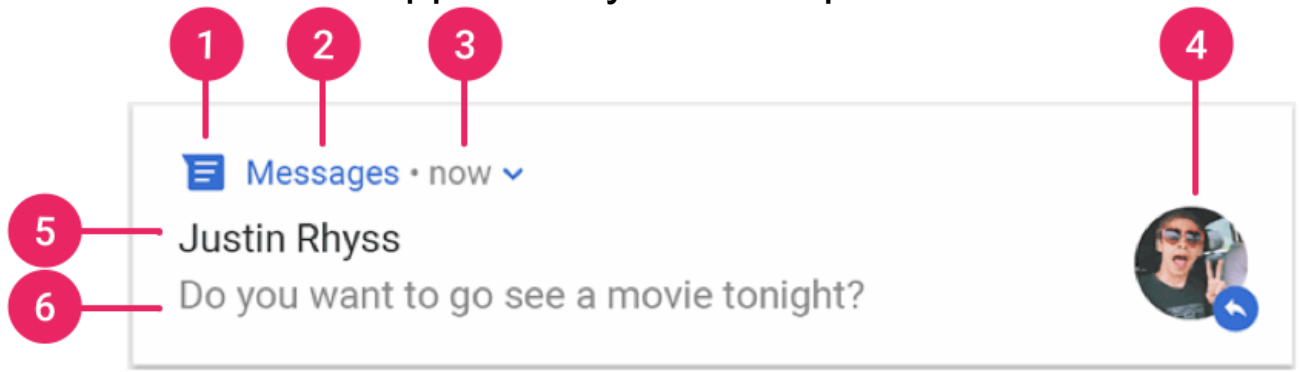
Types of Notifications In Android, there are two main types of notifications:

- **Local Notifications**: These are notifications sent directly from the user's device by the app. Local notifications can be scheduled in advance or sent instantly without requiring an internet connection.
- **Remote Notifications** (also known as Push Notifications): These are notifications sent from a server to the user's device through a third-party service (e.g., Firebase Cloud Messaging). To receive remote notifications from the server, the device must have an internet connection.

4. The structure of the notification

Notification anatomy

The design of a notification is determined by system templates, and your app defines the contents for each portion of the template. Some details of the notification appear only in the expanded view.

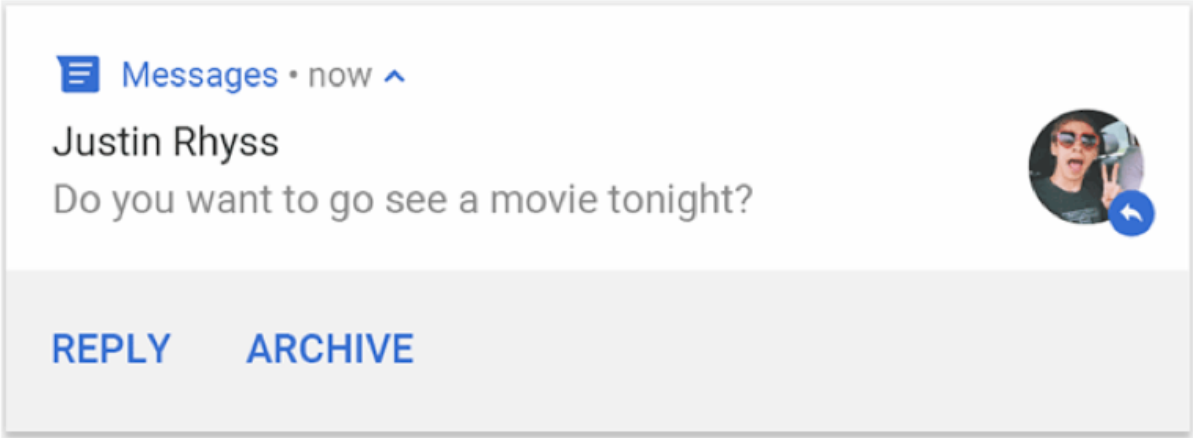


1. Small icon: required; set using setSmallIcon().
2. App name: provided by the system.
3. Time stamp: provided by the system, but you can override it using setWhen() or hide it using setShowWhen(false).
4. Large icon: optional; usually used only for contact photos. Don't use it for your app icon. Set using setLargeIcon().
5. Title: optional; set using setContentTitle().
6. Text: optional; set using setContentText().

4. The structure of the notification

Notification actions

Although it's not required, it's a good practice for every notification to open an appropriate app activity when it's tapped.
In addition to this default notification action,
you can add action buttons that complete an app-related task from the notification—often without opening an activity—as shown in figure 8.

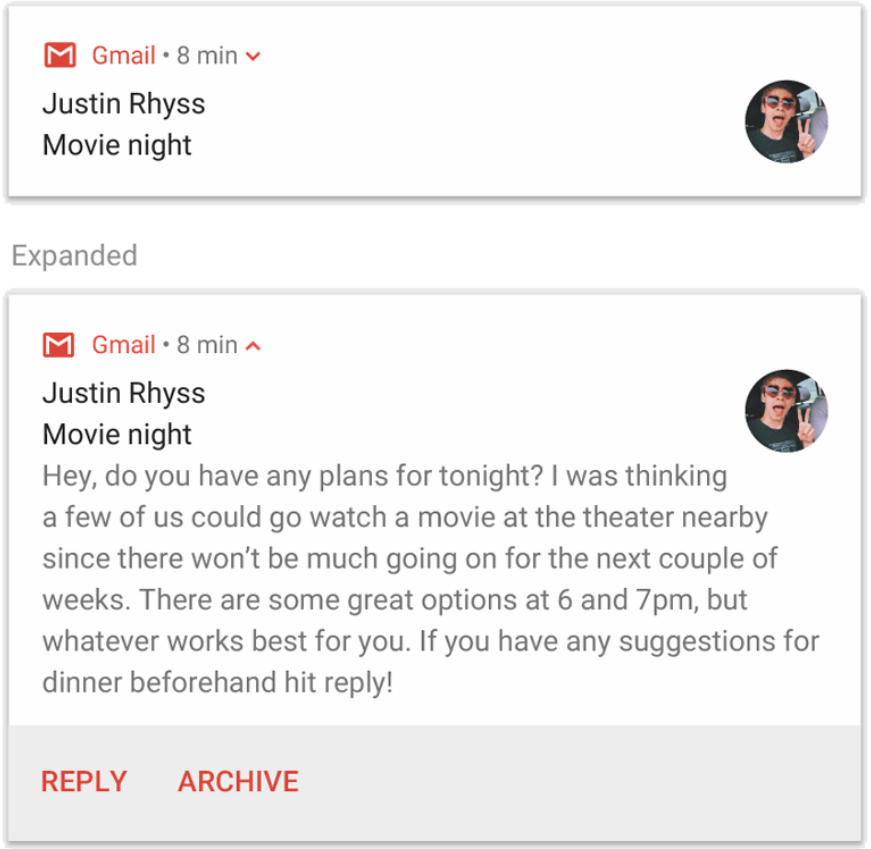


Starting in Android 7.0 (API level 24), you can add an action to reply to messages or enter other text directly from the notification.
Starting in Android 10 (API level 29), the platform can automatically generate action buttons with suggested intent-based actions.

4. The structure of the notification

Expandable notification

By default, the notification's text content is truncated to fit on one line.
If you want your notification to be longer, you can enable a larger text area that's expandable by applying an additional template.

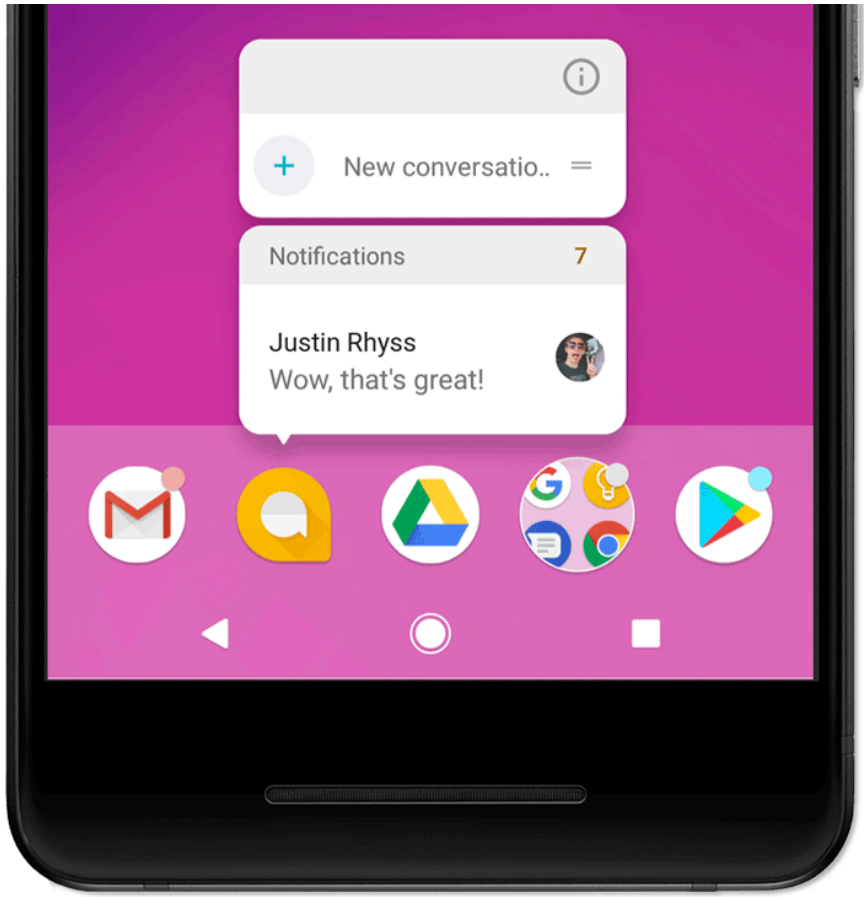


4. The structure of the notification

App icon badge

In supported launchers on devices
running Android 8.0 (API level 26) and higher,
app icons indicate new notifications with a colored badge known as
a notification dot on the corresponding app launcher icon.

Users can touch & hold an app icon
to see the notifications for that app.
Users can dismiss or act on notifications
from that menu, similar to the notification drawer.



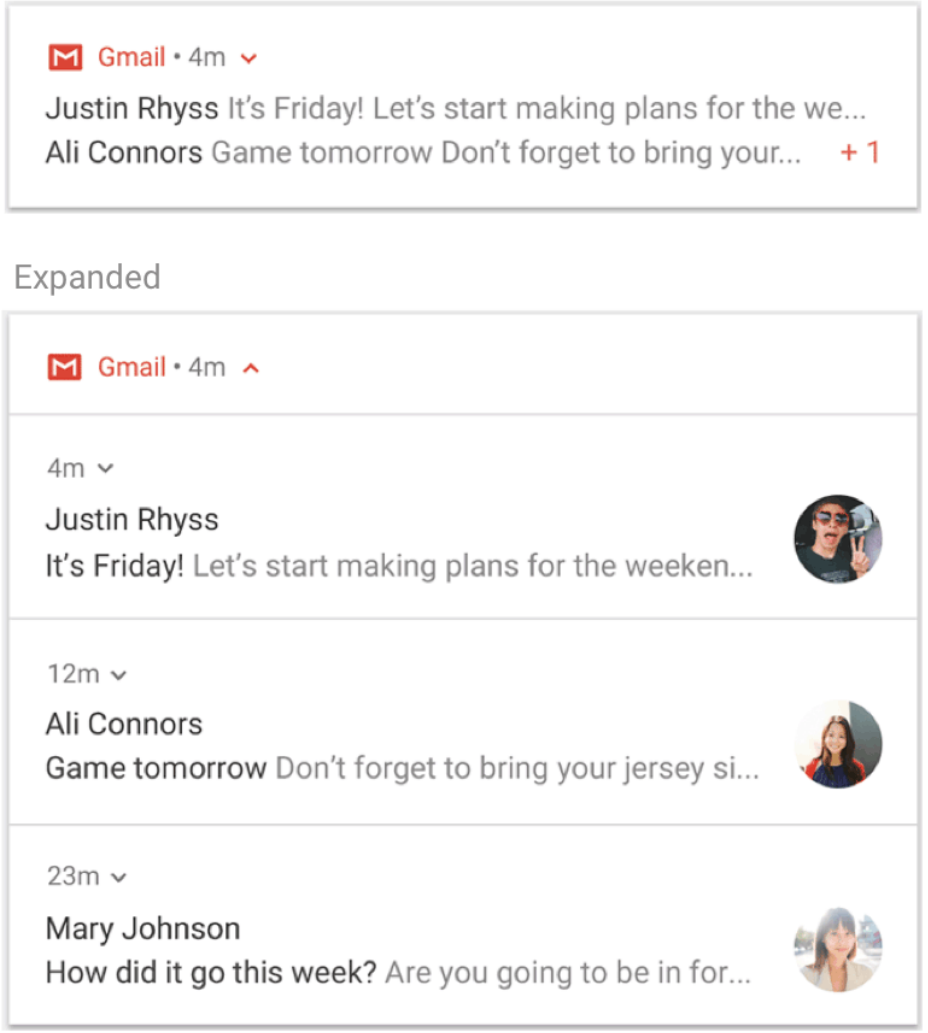
4. The structure of the notification

Notification updates and groups

To avoid flooding your users with multiple or redundant notifications when you have additional updates, update an existing notification rather than issuing a new one or use the inbox-style notification to show conversation updates.

However, if it's necessary to deliver multiple notifications, consider grouping the separate notifications into a group, available on Android 7.0 and higher.

A notification group lets you collapse multiple notifications into one post in the notification drawer with a summary.
The user can progressively expand the notification group and each notification within it for more details.



5. Creating, Managing a Notification

Notification runtime permission

Android 13 (API level 33) and higher supports a runtime permission for sending non-exempt (including Foreground Services (FGS)) notifications from an app: POST_NOTIFICATIONS. This change helps users focus on the notifications that are most important to them.

We highly recommend that you target Android 13 or higher as soon as possible to benefit from the additional control and flexibility of this feature. If you continue to target 12L (API level 32) or lower, you lose some flexibility with requesting the permission in the context of your app's functionality.

```
<manifest ...>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
    <application ...>
        ...
    </application>
</manifest>
```

5. Creating, Managing a Notification

Create a notification channel

To create a notification channel, follow these steps:

Construct a NotificationChannel object with a unique channel ID, user-visible name, and importance level.

Optionally, specify the description that the user sees in the system settings with setDescription().

Register the notification channel by passing it to createNotificationChannel().

```
private void createNotificationChannel() {
    // Create the NotificationChannel, but only on API 26+ because
    // the NotificationChannel class is not in the Support Library.
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = getString(R.string.channel_name);
        String description = getString(R.string.channel_description);
        int importance = NotificationManager.IMPORTANCE_DEFAULT;
        NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name, importance);
        channel.setDescription(description);
        // Register the channel with the system. You can't change the importance
        // or other notification behaviors after this.
        NotificationManager notificationManager = getSystemService(NotificationManager.class);
        notificationManager.createNotificationChannel(channel);
    }
}
```

5. Creating, Managing a Notification

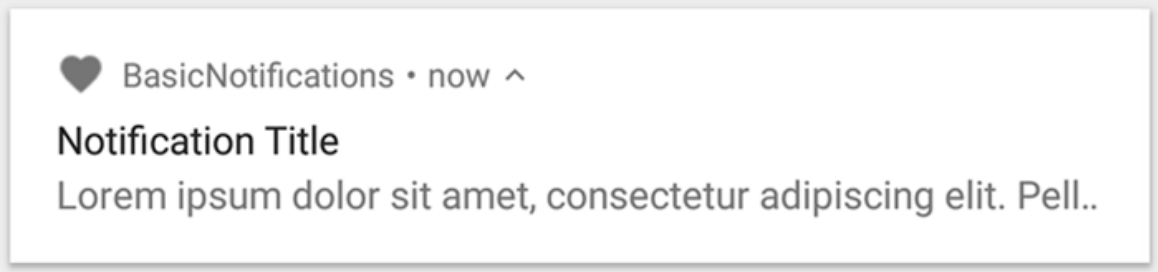
Create a notification channel

By default, all notifications posted to a given channel use the visual and auditory behaviors defined by the importance level from the NotificationManagerCompat class,
such as IMPORTANCE_DEFAULT or IMPORTANCE_HIGH.
See the next section for more information about importance levels

User-visible importance level	Importance (Android 8.0 and higher)	Priority (Android 7.1 and lower)
Urgent Makes a sound and appears as a heads-up notification.	`IMPORTANCE_HIGH`	`PRIORITY_HIGH` or `PRIORITY_MAX`
High Makes a sound.	`IMPORTANCE_DEFAULT`	`PRIORITY_DEFAULT`
Medium Makes no sound.	`IMPORTANCE_LOW`	`PRIORITY_LOW`
Low Makes no sound and doesn't appear in the status bar.	`IMPORTANCE_MIN`	`PRIORITY_MIN`
None Makes no sound and doesn't appear in the status bar or shade.	`IMPORTANCE_NONE`	`N/A`

5. Creating, Managing a Notification

Create a basic notification




```
NotificationCompat.Builder builder = new NotificationCompat.Builder(this, CHANNEL_ID)
        .setSmallIcon(R.drawable.notification_icon)
        .setContentTitle("My notification")
        .setContentText("Much longer text that cannot fit one line...")
        .setStyle(new NotificationCompat.BigTextStyle()
                .bigText("Much longer text that cannot fit one line..."))
        .setPriority(NotificationCompat.PRIORITY_DEFAULT);
```

Set the notification content

To get started, you need to set the notification's content and channel using a NotificationCompat.Builder object.

The following example shows how to create a notification with the following:

A small icon, set by **setSmallIcon().**

This is the only user-visible content that's required.

A title, set by **setContentTitle().**

The body text, set by **setContentText().**

The notification priority, set by **setPriority().**

5. Creating, Managing a Notification

Show the notification

To make the notification appear, call NotificationManagerCompat.notify(), passing it a unique ID for the notification and the result of NotificationCompat.Builder.build(). For example:

```
NotificationManagerCompat notificationManager = NotificationManagerCompat.from(this);

// notificationId is a unique int for each notification that you must define
notificationManager.notify(notificationId, builder.build());
```

THANK YOU!